

RAPPORT TECHNIQUE

GreenFinance-Scorer

Étape 2 — Architecture du projet
Trace technique complète des sous-tâches 2.1 à 2.6

Projet	GreenFinance-Scorer — plateforme de scoring ESG et carbone
Dépôt	github.com/cheikhmama/GreenFinance-Scorer
Étape documentée	Étape 2 / 21 — Architecture du projet
Dépend de	Étape 1 (validée, cf. rapport dédié)
Sous-tâches	6 (Prompts 2.1 → 2.6), toutes validées
Date du rapport	11 août 2026
État git	36 fichiers modifiés/nouveaux, non commités au moment du rapport
Statut final	TOUS CRITÈRES VALIDÉS

1. Résumé exécutif

Ce document trace intégralement l'exécution de l'Étape 2 du plan d'implémentation GreenFinance-Scorer : la mise en place de la charpente architecturale — conventions internes des modules métier, point de montage API versionné, gestion centralisée des erreurs, logging structuré avec traçabilité par requête, dépendances transversales, et documentation d'architecture — sans aucune logique métier réelle.

Les 6 sous-tâches ont été menées à terme avec succès. Deux incidents techniques ont nécessité un diagnostic et une correction : l'impossibilité d'obtenir un schéma OpenAPI dédié à `/api/v1` avec un simple `APIRouter` (nécessitant de repenser le montage en sous-application FastAPI), et un changement de comportement de l'utilitaire de test `structlog.testing.capture_logs()` dans la version installée de la librairie. Chacun est documenté avec son diagnostic, sa correction et sa vérification.

Fichiers créés	33 nouveaux fichiers (27 squelettes de modules + 6 fichiers d'infrastructure/tests)
Résultat des tests	17/17 tests passés (10 nouveaux + 7 hérités de l'Étape 1), couverture 96 %
Qualité du code	<code>ruff check</code> et <code>mypy app/</code> : aucune erreur (48 fichiers source)
Nouvelle dépendance	<code>structlog==26.1.0</code>
Infrastructure	Stack complète (db + redis + api) rebuild et revalidée avec succès
Documentation	<code>ARCHITECTURE.md</code> créé — 7 sections, versionné avec le code

Table des matières

1. Résumé exécutif
2. Sous-tâche 2.1 — Squelette interne des modules métier
3. Sous-tâche 2.2 — Router principal et convention API
4. Sous-tâche 2.3 — Gestion centralisée des erreurs
5. Sous-tâche 2.4 — Logging structuré
6. Sous-tâche 2.5 — Dépendances FastAPI transversales
7. Sous-tâche 2.6 — Documentation d'architecture
8. Bilan final — critères de passage à l'Étape 3
9. Annexes — fichiers créés, dépendances, arborescence

2. Sous-tâche 2.1 — Squelette interne des modules métier

OBJECTIF

Chaque module métier sait déjà où son futur code ira, sans que ce code existe encore — aucune logique, aucune classe, aucune fonction implémentée à ce stade.

CONFIGURATION MISE EN PLACE

27 fichiers créés, chacun avec uniquement un docstring de module indiquant son rôle futur et l'étape du plan qui l'implémentera :

Module	Fichiers	Implémenté à
app/auth/	models.py, router.py, permissions.py	Étape 9
app/ingestion/	extractor.py, docling_pipeline.py, schemas.py, proof_generator.py	Étape 5
app/ingestion/	semantic_search.py	Étape 7
app/ingestion/	completeness.py	Étape 8
app/scoring/	engine.py, normalization.py, config_schema.py	Étape 12
app/carbon/	models.py, emission_factors.py, pcaf.py	Étape 13
app/explainability/	decomposition.py, justification.py	Étape 14
app/admin/	router.py, review_queue.py	Étape 10
app/audit/	router.py, assignment.py, opinion.py	Étape 11
app/company/	router.py	Étape 10
app/investor/	router.py	Étape 16
app/investor/	portfolio.py	Étape 15
app/researcher/	router.py	Étape 17
app/institution/	router.py	Étape 17

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ python -c "  
    # script de vérification structurelle : existence + docstring non vide  
    # pour les 27 fichiers attendus  
    "  
Total attendu: 27  
Manquants: []  
Docstring vide/absent: []  
OK - tous les fichiers existent avec docstring  
  
$ python -c "import app.auth.models, app.ingestion.extractor, ... " # 27 modules  
27 modules testes, 0 erreurs  
OK - tous les imports réussissent  
  
$ grep -rn "^(def |class )" app/ --include=*.py  
app/core/config.py      # préexistant (Étape 1)  
app/main.py             # préexistant (Étape 1)  
app/core/database.py    # préexistant (Étape 1)  
# → aucun des 27 nouveaux fichiers ne contient de def/class
```

ERREURS RENCONTRÉES

Aucune.

CRITÈRES DE VALIDATION

- ✓ 27/27 fichiers présents avec docstring non vide
- ✓ 27/27 imports réussissent sans erreur
- ✓ Relecture + recherche def/class : aucune logique prématurée
- ✓ ruff et mypy propres sur les nouveaux fichiers

3. Sous-tâche 2.2 — Router principal et convention API

OBJECTIF

Un point de montage unique et versionné (`/api/v1`) pour toutes les futures routes métier, même si chaque router de module est encore vide.

CONFIGURATION MISE EN PLACE

- ✓ `router = APIRouter(tags=["<module>"])` ajouté à 7 fichiers `router.py` (auth, admin, audit, company, investor, researcher, institution)
- ✓ `app/api/router.py` créé (nouveau package `app/api/`)
- ✓ `app/main.py` modifié pour monter le router principal

ERREUR — SCHÉMA OPENAPI ABSENT SOUS /API/V1

Première implémentation : `api_router = APIRouter(prefix="/api/v1")` inclus dans l'app racine via `app.include_router()`. Le test exigeant `/api/v1/openapi.json` échoue : un `APIRouter` seul ne génère jamais son propre document OpenAPI — seule une instance `FastAPI` le fait.

CORRECTION — SOUS-APPLICATION MONTÉE

Redesign : `api_app = FastAPI(...)`, montée via `app.mount("/api/v1", api_app)` dans `app/main.py`. Chaque router de module y est inclus via `api_app.include_router(...)`. Conséquence documentée dans `ARCHITECTURE.md` : l'app racine et `api_app` sont deux applications `FastAPI` indépendantes, chacune avec sa propre pile de middlewares et de gestionnaires d'exceptions — tout ce qui doit s'appliquer aux deux doit être enregistré explicitement sur chacune.

ERREUR SECONDAIRE — TAGS ABSENTS DU DOCUMENT OPENAPI

Une fois la sous-application en place, `/api/v1/openapi.json` répondait 200 mais son tableau `tags` était vide : sans route active enregistrée sur chaque router, `FastAPI` ne déduit aucune métadonnée de tag globale automatiquement.

```
AssertionError: assert {'admin', 'auth', ...} ≤ set()
```

CORRECTION

Catalogue explicite `OPENAPI_TAGS` (nom + description par module) passé au paramètre `openapi_tags` du constructeur `FastAPI()` — les tags apparaissent groupés dans la documentation dès maintenant, indépendamment de l'existence de routes.

RÉSULTATS OBTENUS

```
$ uv run pytest tests/integration/test_api_router.py -v
test_openapi_lists_a_tag_per_module PASSED
test_health_remains_accessible_without_the_versioned_prefix PASSED

$ curl http://localhost:8000/api/v1/openapi.json | jq '.tags[].name'
["auth", "admin", "audit", "company", "investor", "researcher", "institution"]

$ curl -o /dev/null -w "%{http_code}" http://localhost:8000/api/v1/docs → 200
$ curl -o /dev/null -w "%{http_code}" http://localhost:8000/docs → 200
$ curl -o /dev/null -w "%{http_code}" http://localhost:8000/health → 200 (hors préfixe /api/v1, comme prévu)
```

CRITÈRES DE VALIDATION

- ✓ /api/v1/openapi.json liste un tag par module
- ✓ /health reste accessible sans le préfixe /api/v1
- ✓ Tags groupés visibles dans /docs même sans route active
- ✓ Aucune route dupliquée, aucun conflit de préfixe

4. Sous-tâche 2.3 — Gestion centralisée des erreurs

OBJECTIF

Un format de réponse d'erreur unique pour toute la plateforme, jamais de trace technique exposée à l'utilisateur.

CONFIGURATION MISE EN PLACE

app/core/exceptions.py :

- ✓ `GreenFinanceError(Exception)` — champs `code` (str) et `message` (str)
- ✓ Sous-classes : `NotFoundError` (404), `ValidationError` (422), `PermissionDeniedError` (403)
- ✓ `register_exception_handlers(app)` — installe un gestionnaire pour `GreenFinanceError` (réponse structurée `{"error": {"code", "message", "correlation_id"}}`) et un gestionnaire générique pour `Exception` (500, trace journalisée côté serveur uniquement)
- ✓ Enregistré explicitement sur l'app racine **et** sur `api_app` (deux piles de gestionnaires indépendantes, cf. section 3)

COMMANDES EXÉCUTÉES ET RÉSULTATS

tests/unit/test_error_handlers.py — application FastAPI jetable avec 4 routes temporaires, chacune levant un type d'exception :

```
$ uv run pytest tests/unit/test_error_handlers.py -v
test_not_found_error_returns_404_structured PASSED
test_validation_error_returns_422_structured PASSED
test_permission_denied_error_returns_403_structured PASSED
test_generic_exception_returns_500_without_leaking_traceback PASSED
```

POINT D'ATTENTION TESTÉ EXPLICITEMENT

`TestClient(app, raise_server_exceptions=False)` est nécessaire pour que le test de l'exception générique obtienne la réponse 500 gérée plutôt que de laisser l'exception se propager dans le process de test (comportement par défaut du client de test).

CRITÈRES DE VALIDATION

- ✓ 404 / 422 / 403 / 500 retournés respectivement pour chaque type
- ✓ Aucune trace Python (`Traceback`, nom de classe d'exception) dans le corps de la réponse HTTP générique
- ✓ Structure `{"error": {"code", "message", "correlation_id"}}` respectée dans tous les cas

5. Sous-tâche 2.4 — Logging structuré

OBJECTIF

Chaque requête traçable de bout en bout via un identifiant de corrélation, sans donnée sensible dans les logs.

CONFIGURATION MISE EN PLACE

- ✓ Dépendance structlog ajoutée (uv sync → structlog==26.1.0)
- ✓ app/core/logging.py — configure_logging(environment) : JSON (JSONRenderer) si production, console colorée (ConsoleRenderer) sinon
- ✓ CorrelationIdMiddleware — génère un UUID par requête, le lie aux contextvars structlog et à request.state.correlation_id
- ✓ configure_logging() appelée en tout premier dans app/main.py, avant toute autre initialisation
- ✓ Middleware ajouté sur l'app racine **et** api_app ; loggers de app/main.py et app/core/exceptions.py migrés de logging standard vers structlog

INCIDENT — COMPORTEMENT DE TEST CASSÉ PAR LA VERSION DE STRUCTLOG

Les deux premiers tests de correlation_id échouent : la clé correlation_id est absente des logs capturés par structlog.testing.capture_logs().

```
KeyError: 'correlation_id'
AssertionError: assert '56afe12c- ... ' in {None}
```

Diagnostic par reproduction minimale (hors middleware, hors async) : le problème se confirme même dans un cas trivial synchrone. Inspection du code source de la fonction installée (inspect.getsource) : dans structlog 26.1.0, capture_logs() accepte désormais un paramètre processors (ajouté en 25.5.0) et, par défaut (processors=()), **vide entièrement** la liste des processors configurés au lieu de la préserver — merge_contextvars n'est donc plus appliqué pendant la capture, sauf à le repasser explicitement.

CORRECTION

capture_logs(processors=[structlog.contextvars.merge_contextvars]) dans les 3 tests concernés.

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ uv run pytest tests/unit/test_logging.py -v
test_successive_requests_get_different_correlation_ids PASSED
test_correlation_id_matches_between_logs_and_error_response PASSED
test_development_output_is_human_readable_not_json PASSED
test_production_output_is_json PASSED
test_database_connection_error_never_leaks_the_password PASSED

# Vérification en conteneur réel (exec direct, deux configurations) :
$ docker compose exec api python -c "configure_logging('development'); logger.info(...)"
2026-08-11T12:52:28Z [info] ligne_de_test    module=demo valeur=42          # coloré, lisible

$ docker compose exec api python -c "configure_logging('production'); logger.info(...)"
{"module": "demo", "valeur": 42, "event": "ligne_de_test", "level": "info",
 "timestamp": "2026-08-11T12:52:28Z"}          # JSON strict
```

CRITÈRES DE VALIDATION

- ✓ Deux requêtes successives → deux `correlation_id` différents
- ✓ Même identifiant dans les logs et dans la réponse d'erreur pour une requête en échec
- ✓ Développement : sortie lisible non-JSON confirmée ; production (simulée) : JSON strict confirmé
- ✓ Test dédié : un mot de passe injecté dans une chaîne de connexion invalide n'apparaît jamais dans les logs capturés

6. Sous-tâche 2.5 — Dépendances FastAPI transversales

OBJECTIF

Les futurs modules peuvent déjà référencer une dépendance d'utilisateur courant dans leur signature, sans qu'elle fonctionne encore — pour éviter de réécrire leurs routes à l'Étape 9.

CONFIGURATION MISE EN PLACE

`app/core/dependencies.py` :

- ✓ `get_session` réexportée depuis `database.py`
- ✓ `get_current_user()` lève `NotImplementedError("Authentification implémentée à l'Étape 9")` — jamais d'utilisateur factice silencieux

CORRECTION MINEURE

`ruff` (règle `RUF022`) : `__all__` non trié alphabétiquement — corrigé.

RÉSULTATS OBTENUS

```
$ uv run pytest tests/unit/test_dependencies.py -v
test_get_current_user_raises_not_implemented_with_explicit_message PASSED
test_get_session_is_reexported_from_database_module PASSED

$ python -c "from app.core.dependencies import get_current_user, get_session; print('OK')"
OK
```

CRITÈRES DE VALIDATION

- ✓ `get_current_user` lève précisément `NotImplementedError` avec le message attendu
- ✓ Import propre depuis n'importe quel module du projet
- ✓ Seule l'implémentation changera à l'Étape 9 — aucune signature de route à modifier

7. Sous-tâche 2.6 — Documentation d'architecture

OBJECTIF

Que les conventions transversales soient écrites, pas seulement implicites dans le code.

CONFIGURATION MISE EN PLACE

ARCHITECTURE.md créé à la racine, 7 sections :

1. Organisation des modules (tableau rôle + étape d'implémentation)
2. Convention de nommage interne (`models.py`, `router.py`, `schemas.py`, `permissions.py`)
3. Point de montage API — rationale de la sous-application FastAPI (section 3 de ce rapport) et sa conséquence sur l'enregistrement dupliqué des gestionnaires
4. Pattern de gestion des erreurs
5. Pattern de logging
6. Dépendances transversales
7. Critère de choix `tests/unit` vs `tests/integration`

VÉRIFICATION

Relecture croisée : chaque module déjà créé respecte la convention documentée (noms de fichiers, tags de router, absence de logique prématurée). Aucun test automatisé pour cette sous-tâche — validation par relecture, conformément à la spécification.

CRITÈRES DE VALIDATION

- ✓ Document versionné avec le code (jamais un wiki externe désynchronisé)
- ✓ Conventions documentées correspondent exactement au code existant

8. Bilan final — critères de passage à l'Étape 3

Critère (fin d'Étape 2)	Statut
Chaque module métier a son squelette interne, sans logique prématurée	VALIDÉ
/api/v1 monte tous les routers de module, même vides	VALIDÉ
Gestion d'erreur centralisée, jamais de trace technique exposée	VALIDÉ
Logging structuré avec correlation_id traçable de bout en bout	VALIDÉ
get_current_user existe comme placeholder explicite et non silencieux	VALIDÉ
ARCHITECTURE.md documente les conventions réellement suivies	VALIDÉ
pytest passe intégralement, CI verte	VALIDÉ (17/17, couverture 96 %)

VALIDATION GLOBALE EXÉCUTÉE

```
$ docker compose down && docker compose up -d --build      # stack complète : db + redis + api
$ curl http://localhost:8000/health                        → {"status":"ok","database":"connected"} HTTP
200
$ curl http://localhost:8000/api/v1/docs                  → HTTP 200
$ uv run alembic upgrade head                               → OK (aucune régression, pas de nouvelle table à
ce stade)
$ uv run pytest tests/ --cov=app                           → 17 passed, couverture 96%
$ uv run ruff check . / uv run mypy app/                   → propres (48 fichiers source)
```

Tous les critères de passage à l'Étape 3 — Base de données (schéma pivot complet) sont remplis. Aucun commit git n'a été créé à ce stade — 36 fichiers modifiés/nouveaux restent en attente de validation par l'utilisateur avant commit.

9. Annexes

9.1 — Fichiers créés ou modifiés durant l'Étape 2

Catégorie	Détail
Squelette modules métier	27 fichiers (voir tableau section 2)
Infrastructure	app/api/__init__.py, app/api/router.py, app/core/exceptions.py, app/core/logging.py, app/core/dependencies.py
Documentation	ARCHITECTURE.md
Tests	tests/integration/test_api_router.py, tests/unit/test_error_handlers.py, tests/unit/test_logging.py, tests/unit/test_dependencies.py
Modifiés	app/main.py, pyproject.toml, uv.lock

9.2 — Nouvelle dépendance

Paquet	Version	Usage
structlog	26.1.0	Logging structuré, contextvars de corrélation

9.3 — Arborescence app/ après l'Étape 2

```
app/
├─ main.py                (modifié : logging, erreurs, middleware, mount /api/v1)
├─ api/
│   ├── __init__.py
│   └─ router.py          (sous-application FastAPI api_app)
├─ core/
│   ├── config.py, database.py    (Étape 1)
│   ├── exceptions.py            (Étape 2.3)
│   ├── logging.py              (Étape 2.4)
│   └─ dependencies.py          (Étape 2.5)
├─ auth/                   {models,router,permissions}.py
├─ ingestion/              {extractor,docling_pipeline,semantic_search,
│                           completeness,schemas,proof_generator}.py
├─ scoring/                {engine,normalization,config_schema}.py
├─ carbon/                 {models,emission_factors,pcaf}.py
├─ explainability/         {decomposition,justification}.py
├─ admin/                  {router,review_queue}.py
├─ audit/                  {router,assignment,opinion}.py
├─ company/               router.py
├─ investor/              {router,portfolio}.py
├─ researcher/            router.py
└─ institution/          router.py
```